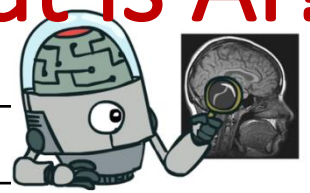
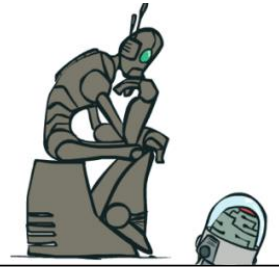
CSE358

ARTIFICIAL INTELLIGENCE

02. Intelligent Agents

Review: What is AI?





Thinking Humanly (1)	Thinking Rationally (3)
"The exciting new effort to make computers think … machines with minds, in the full and literal sense." (Haugeland, 1985) **machines with minds** **Cognitive Approach** "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning …" (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) **involves using math and logic to do AI** **Law of thoughts** "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992) **codifying the right thinking with logic**
Acting Humanly (2)	**Acting Rationally → OUR APPROACH (4)**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) **machines do things just like humans** "The study of how to make computers do things which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) **machines to act rationally rather than humanly** "AI … is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

Four schools of thoughts (Russel & Norvig)

Review: Acting Rationally – Rational Agents

machines to act rationally rather than humanly

- **Rational behavior: doing the "right thing"**
  - The ***right thing***: which is expected to maximize goal achievement, given the available information about the environment
  - Doesn't necessarily involve thinking, e.g., blinking reflex
  - Thinking should be in the service of rational action
  - Entirely dependent on goals!
  - Rational ≠ successful OR Irrational ≠ insane, irrationality is sub-optimal action
- **Our focus here: Rational Agents**
  - A **rational agent** is one that acts to achieve the best outcome, or when there is uncertainty, the <u>best expected outcome</u>
  - Systems which make the best possible decisions given goals, evidence, and constraints
  - In the real world, usually lots of uncertainty … and lots of complexity
  - Usually, we're just approximating rationality
- Aristotle (Nicomachean Ethics):

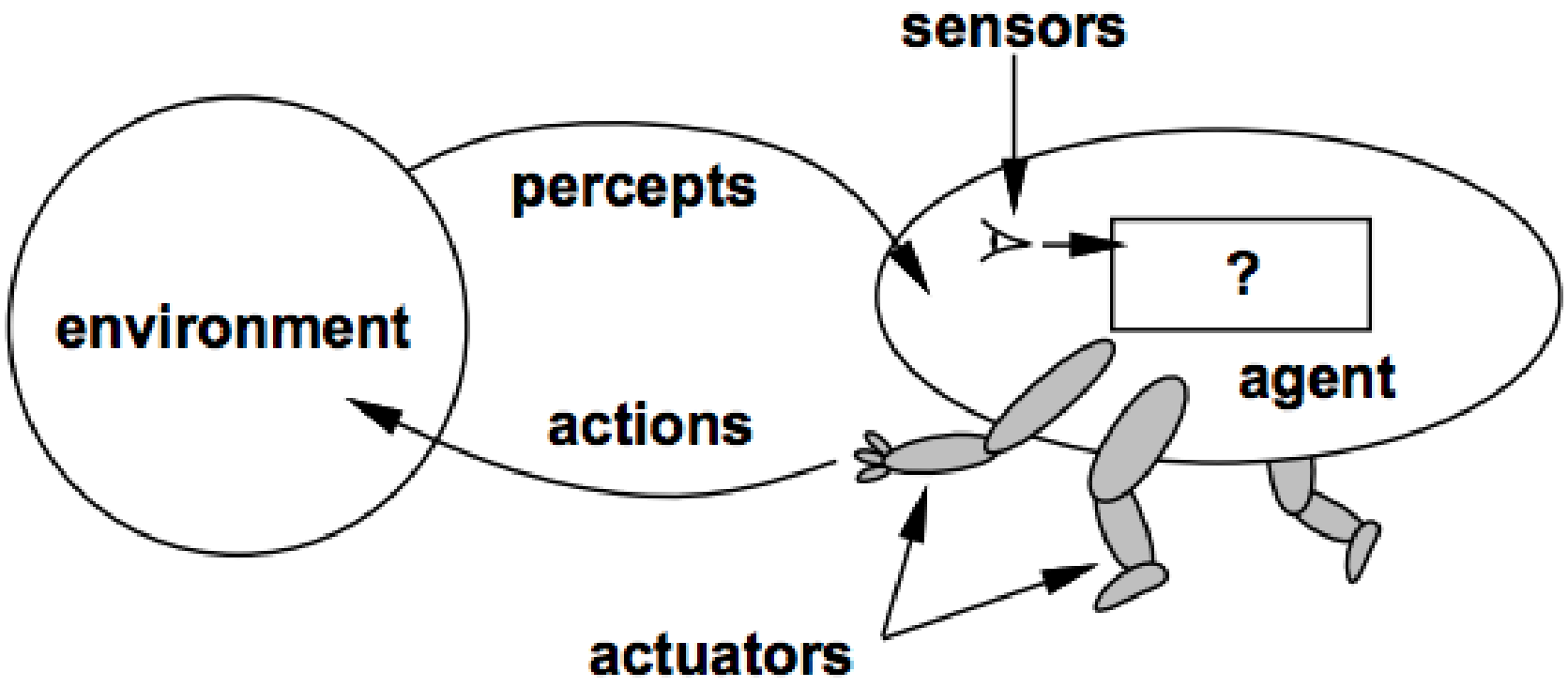
"Every art and every inquiry, and similarly every action and pursuit, is thought to <u>aim at some good</u>."

Outline

- Agents and Environments
- *Good Behavior:* The Concept of Rationality
- The Nature of Environments
  - PEAS
  - Environment Types
- The Structure of Agents
  - Agent Types
  - How the components of agent programs work?

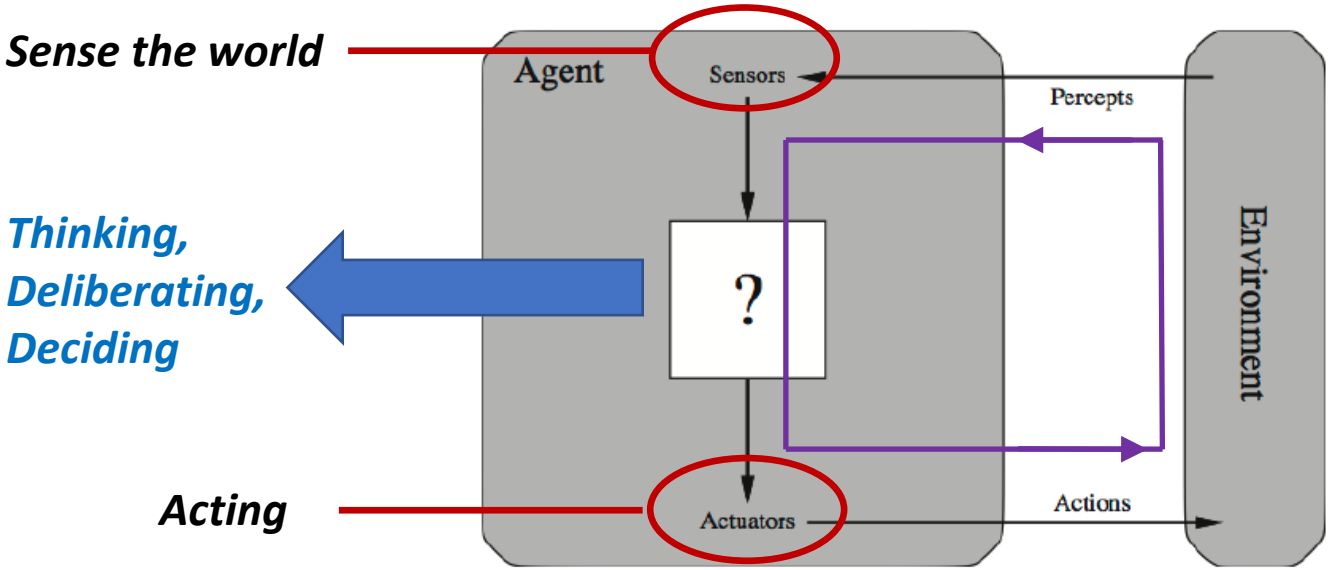
Agents and Environments

- **Agent:** An **agent** is anything that can be viewed as
  - **perceiving** its **environment** through **sensors** and
  - **acting** upon that **environment** through **actuators**



Agents and Environments

- **Agent:** An **agent** is anything that can be viewed as
  - **perceiving** its **environment** through **sensors** and
  - **acting** upon that **environment** through **actuators**



- An agent program runs in cycles of: **(1)perceive**, **(2)think**, and **(3)act**
- **Agent = Architecture + Program**

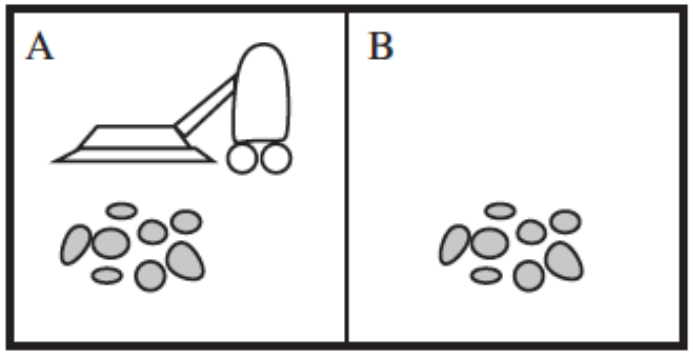
(complimentary and compatible)

Agents and Environments

- **Human Agents**
  - Sensors: eyes, ears, and other organs
  - Actuators: hands, legs, mouth, and other body parts
- **Robotic Agents**
  - Sensors: cameras and infrared range finders
  - Actuators: various motors
- **Software Agents**
  - Sensors: keystrokes, file contents, network packages
  - Actuators: displaying on the screen, writing files, sending network packages

- ***Agents are everywhere!***
  - Thermostat
  - Cell phone
  - Vacuum cleaner
  - Robot
  - Alexa Echo
  - Self-driving car
  - Human
  - etc.

Vacuum Cleaner



- Percepts: location and contents, e.g., [A, Dirty]
- Actions: *Left*, *Right*, *Suck*, *NoOp*
- Agent function: mapping from percepts to actions $f : P^* \rightarrow A$

Percept	Action
[A, clean]	*Right*
[A, dirty]	*Suck*
[B, clean]	*Left*
[B, dirty]	*Suck*

[A, clean], [A, clean]	*Right*
[A, clean], [A, dirty]	*Suck*

What is the right way to fill out the table?

What makes an agent good or bad, intelligent or stupid?

Good Behavior: The concept of rationality

- **Recall:** A **rational agent** is one that **does the right thing**
  - every entry in the table for the agent function is filled out correctly
  - doing the right thing is better than doing the wrong thing …

- … But what does it mean to do the right thing?

  Consider the *consequences* of the agent's behavior

  When an agent is plunked down in an environment, it generates a **sequence of actions** according to the percepts it receives. This sequence of actions causes the environment to go through a **sequence of states**.

  If the sequence is desirable, then the agent has **performed well** → This notion of desirability is captured by a **performance measure** that evaluates any given sequence of **environment states**

Good Behavior: do the right thing

- What are the ***consequences*** of the agent's actions on the ***state*** of its ***environment***?
  - Sequence of agent actions → sequence of environmental states
- How to control agent?
  - Introduce **performance measure**
  - So, rationality is relative to a **performance measure** - ***an objective criterion for success of an agent's behavior***
- Evaluate performance of environment or that of agent?
  - Notice that we said ***environment* states**, not ***agent* states**
  - If we define success in terms of agent's opinion of its own performance, an agent could achieve perfect rationality simply by deluding itself that its performance was perfect

Example: Performance Measure

- Possible performance measures for vacuum world?
  - Amount of dirt cleaned up in an 8-hour shift
  - +1 point for each clean square at each time step
  - +1 point for clean square, -1 for each move
  - The time required for clean-up
  - The degree of noise by the motor of the cleaner
  - Penalty for electricity consumed

- No one fixed performance measure for all tasks and agents

As a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave

Definition of a Rational Agent

Rational Agent

"For each possible percept sequence, a rational agent should select an action that is expected to maximize its ***performance measure****, given the evidence provided by the <u>percept sequence</u> and whatever built-in <u>knowledge</u> the agent has."*

The Concept of Rationality

- Define rationality relative to a **performance measure**
- Judge rationality based on:
  - The performance measure that defines the criterion of success
    - Performance measure awards one point for each clean square at each time step, over a "lifetime" of 1000 time steps.
  - The agent prior knowledge of the environment
    - The "geography" of the environment is known a priori but the dirt distribution and the initial location of the agent are not.
  - The possible actions that the agent can perform
    - Left , Right , and Suck.
  - The agent's percept sequence to date
    - The agent correctly perceives its location and whether that location contains dirt.

Task Environment: PEAS

- When we define a rational agent, we group these properties under **PEAS**, which is actually ***the problem specification for the task environment that the rational agent is meant to solve***
- Task environments are essentially the "problems" to which rational agent that we want to design are the "solutions" (for this task environment)
- PEAS stands for:
  - ✓ **P**erformance
  - ✓ **E**nvironment
  - ✓ **A**ctuators
  - ✓ **S**ensors

In designing an intelligent agent, the first step must always be to specify the task environment as fully as possible

Example: PEAS – self-driving car

What is PEAS for a **self-driving car**?

- **P**erformance: Safety, time, legal drive, comfort
- **E**nvironment: Roads, other cars, pedestrians, road signs
- **A**ctuators: Steering, accelerator, brake, signal, horn
- **S**ensors: Camera, sonar, GPS, Speedometer, odometer, accelerometer, engine sensors, keyboard

Example: PEAS – vacuum cleaner

What is PEAS for a **vacuum cleaner**?

Neato Botvac Series

- **P**erformance: Cleanness, efficiency (distance traveled to clean), battery life, security
- **E**nvironment: Room, table, wood floor, carpet, different obstacles
- **A**ctuators: Wheels, different brushes, vacuum extractor
- **S**ensors: Camera, dirt detection sensor, cliff sensor, bump sensors, infrared wall sensors

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Properties of Task Environments

- The nature of environment can dramatically impact the complexity of the required agent program as well as the difficulty of achieving goals in it

- Types of Task Environments
  - ✓ Fully Observable vs. Partially Observable
  - ✓ Single-agent vs. Multi-agent
  - ✓ Deterministic vs. Stochastic
  - ✓ Episodic vs. Sequential
  - ✓ Static vs. Dynamic
  - ✓ Discrete vs. Continuous
  - ✓ Known vs. Unknown

Fully Observable vs. Partially Observable

Can the agent observe the complete state of the environment?

- **Fully Observable:** An agent's sensors give it access to the complete state of the environment at all times (the full picture)
  - Practical, because the agent need not maintain any internal state to keep track of the world
- **Partially Observable:** An environment might be partially observable due to noisy and inaccurate sensors, or because parts of the state are simply missing from the sensor data
- *Example*: Vacuum Cleaner (room) vs. Self-driving Car (street)

  Perfect GPS vs. noisy pose estimation

Almost everything in the real world is partially observable

Single-agent vs. Multi-agent

- **Single-agent:** An agent operating by itself in an environment
  - *Example*: an agent solving a crossword puzzle by itself is a single-agent
- **Multi-agent:**
  - *Example*: an agent playing chess is in a two-agent environment
  - **Competitive Multi-agent**: If the agents are maximizing a performance metric that depends on other agents' behavior
    - *Example*: In chess, the opponent entity B is trying to maximize its performance measure, which, by the rules of chess, minimizes agent A's performance measure. Thus, chess is a **competitive** multi-agent environment

Deterministic vs. Stochastic

Is there uncertainty in how the world works?

- **Deterministic:** The next state of the environment is completely determined by the current state, and the action executed by the agent
  - If the environment is deterministic except for the actions of the other agents, then the environment is called **strategic**
  - In principle, an agent need not worry about uncertainty in a fully observable, deterministic environment
  - *Example*: The vacuum cleaner world is deterministic
- **Stochastic:** Opposite of deterministic is non-deterministic, or stochastic, in which uncertainty about outcomes is modeled or quantified in terms of probabilities
  - Actions have probabilistic outcomes - we have this possibility of making this action for the agent
  - If the environment is partially observable, then it could ***appear*** to be stochastic – Strongly related to ***partial observability***
  - *Example:* Basically, a car can't predict the behavior of other cars, or the traffic conditions, moreover, one's tires blow out and one's engine seizes up without warning

Episodic vs. Sequential

- **Episodic:** The agent's experience is divided into atomic "episodes" (each episode consists of the agent perceiving and then performing a single action), and the choice of action in each episode depends only on the episode itself (in other words, the next episode does not depend on the actions taken in previous ones)
  - *Example:* Many classification tasks are episodic
- **Sequential:** current decision could affect all future decisions
  - *Example:* Chess (short-term actions can have long-term consequences)
- Episodic environments are much simpler than sequential environments because the agent does not need to think ahead

Static vs. Dynamic

- **Static:** The environment is unchanged while an agent is deliberating or thinking
  - Static environments are easy to deal with because the agent need not keep looking at the world while it is deciding on an action, nor need it worry about the passage of time
  - *Example:* Crossword puzzles
- **Dynamic:** The environment is continuously asking the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing
  - *Example:* Driving car
  - The environment is **semi-dynamic** if the environment itself does not change with the passage of time but the agent's performance score does
    - *Example:* Chess, when played with a clock is semi-dynamic

Discrete vs. Continuous

- The discrete/continuous distinction applies to the *state* of the environment, to the way *time* is handled, and to the *percepts* and *actions* of the agent
- **Discrete:** A limited number of distinct, clearly defined percepts and actions
  - E.g., chess is an example of a discrete environment
    - the chess environment has a finite number of distinct states (excluding the clock)
    - Chess also has a discrete set of percepts and actions
  - E.g., while self-driving car evolves in a continuous one, in terms of both the actions and the time
    - Self-driving car is a continuous-state and continuous-time problem: the speed and location of the car and of the other vehicles sweep through a range of continuous values and do so smoothly over time

Known vs. Unknown

- Known vs. unknown distinction refers not to the environment itself, but to the agent's (or designer's) state of knowledge about the "rules of the game / laws of physics" of the environment
  - The designer of the agent may or may not have knowledge about the environment makeup
- In a **known** environment, the outcomes for all actions are given
- If the environment is **unknown**, the agent will have to learn how it works in order to make good decisions
- ***Note***: the distinction between known and unknown environments is not the same as the one between fully and partially observable environments
  - It is quite possible for a *known* environment to be *partially* observable—for example, in solitaire card games, I know the rules but I am still unable to see the cards that have not yet been turned over
  - Conversely, an *unknown* environment can be *fully* observable—in a new video game, the screen may show the entire game state but I still don't know what the buttons do until I try them

Example: Environment Types

Environment	Observable	Agents	Deterministic	Static	Discrete
8-puzzle	Fully	Single	Deterministic	Static	Discrete
Chess	Fully	Multi	Deterministic	(Semi)Static	Discrete
Pocker	Partially	Multi	Stochastic	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Static	Discrete
Car	Partially	Multi	Stochastic	Dynamic	Continuous
Vacuum cleaner	Partially	Single	Stochastic	Dynamic	Continuous

The environment type largely determines the agent design

The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Agent Functions and Agent Programs

- An agent is completely specified by the **agent function**
  - The **agent function** maps from ***<u>percept sequences</u>*** to ***<u>actions</u>***
  - So, describes agent's behavior

$$\boldsymbol{f} : P^* \rightarrow A$$

abstract mathematical description

  - **Percept:** refers to the agent's perceptual inputs at any given instant
    - An agent's **percept sequence** is the complete history of everything the agent has ever perceived – or **percept history**
    - In general, *an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived*

Agent Functions and Agent Programs

- An agent is completely specified by the **agent function**
  - The **agent function** maps from ***percept sequences*** to ***actions***

$$\boldsymbol{f} : P^* \rightarrow A$$

abstract mathematical description

- ***Job of AI***: Design an **agent program** which implements the **agent function**
  - The **agent program** runs on the physical **architecture** to produce the **agent function**, $\boldsymbol{f}$

concrete implementation, running within some physical system

  - Describes how the agent works
  - Agent function: Mathematical abstraction
  - Agent program: Concrete implementation

Agent = Architecture + Program

- Program must suit architecture
- ***Aim:*** Find a way to implement the rational agent function concisely

Table Lookup Agent

Percept sequence	Action
[A, clean]	*Right*
[A, dirty]	*Suck*
[B, clean]	*Left*
[B, dirty]	*Suck*
[A, clean], [A, clean]	*Right*
[A, clean], [A, dirty]	*Suck*
…	…

- Drawbacks:
  - Huge table
  - Take a long time to build the table
  - No autonomy
  - Even with learning, need a long time to learn the table entries

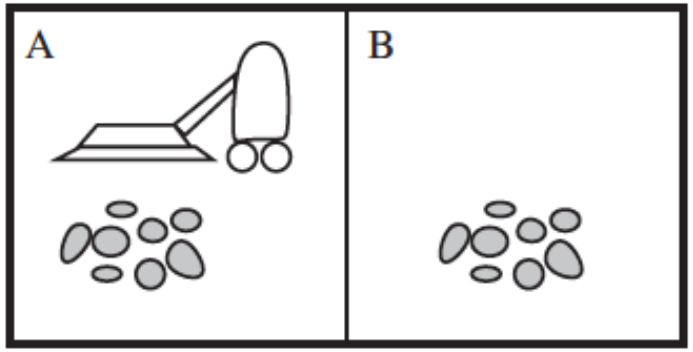


Table-driven-agent

- Let P be the set of possible percepts and let T be the lifetime of the agent the lookup table will contain $\sum_{t=1}^{T} |P|^T$ entries.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
   persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

   append percept to the end of percepts
   action ← LOOKUP(percepts, table)
   return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Table Lookup Agent

- Why a chess table agent is a bad idea:
  - Not enough space in universe to store table (10^{150} entries)
    - the number of atoms in the observable universe is less than 10^{80}
  - Would take longer than your lifetime to build
  - Too massive to be learned through observation
  - Where do you start?

- <u>Reminder:</u> The fact that a program can find a solution in principle does not mean that the program contains any of the mechanisms needed to find it in practice

Can we do better?

- Better is to the extent possible, produce rational behavior from a smallish program rather than from a vast table.

- Example: five-line program for Newton's method running on electronic calculators.

Let N be any number, then the square root of N can be given by the formula:

$root = 0.5 * (X + (N/X))$ where X is any guess which can be assumed to be N or 1.

1. Assign X to the N itself.
2. Now, start a loop and keep calculating the root which will surely move towards the correct square root of N.
3. Check for the difference between the assumed X and calculated root, if not yet inside tolerance, then update root and continue.
4. If the calculated root comes inside the tolerance allowed, then break out of the loop.
5. Print the root.

Agent Types

- There are 4 types of agents in general, varying in ***the level of intelligence*** or the ***complexity of the tasks they are able to perform***

- Four basic types in order of <u>increasing generality</u>
  1. **Simple reflex** agents
  2. **Model-based** reflex agents
  3. **Goal-based** agents
  4. **Utility-based** agents

- All of which can be generalized into **learning agents** that can improve their performance and generate better actions

1. Simple Reflex Agents

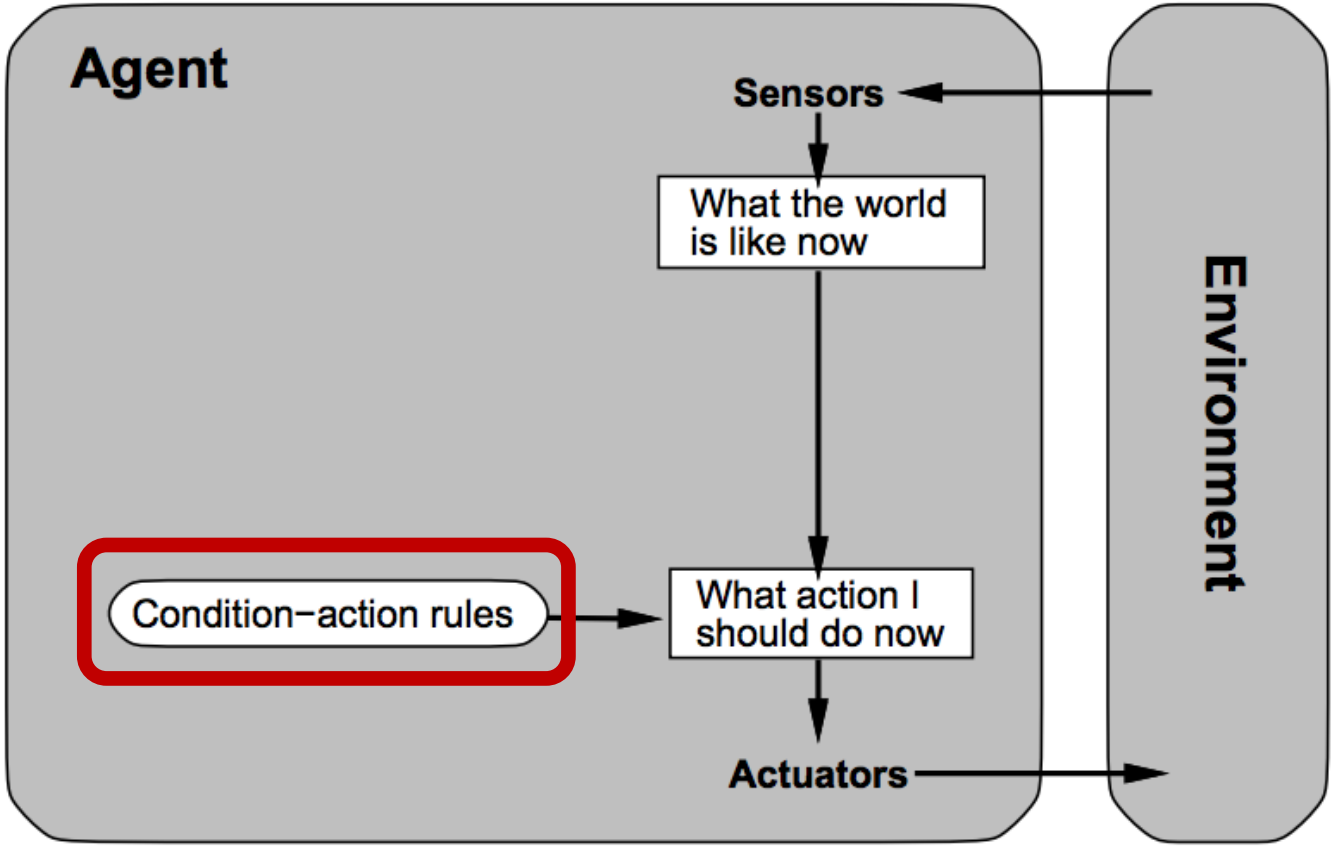
- Forget the percept history!
- Can't we just act on the current state?
  - Select actions based on the current percept ignoring the rest of the percept history

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

1. Simple Reflex Agents

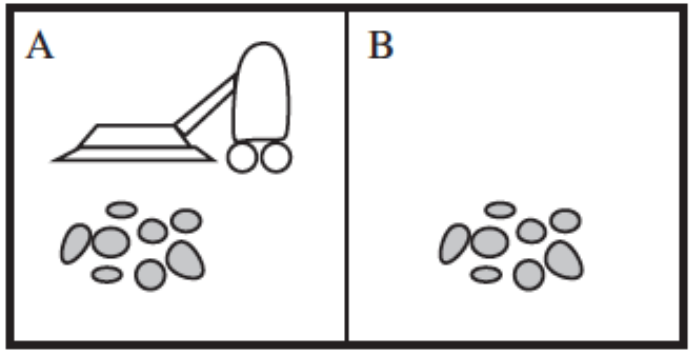
- Simple reflex agents select an action based on **the current state only**, ignoring the percept history
- Simple, but turn out to be of very limited intelligence
- Can only work if the environment is **fully observable**, that is **the correct action is based on what is perceived currently** – **based on the current percept only**



Condition-action rule

if *car-in-front-is-braking* **then** *initiate-braking*

Example: Vacuum (Reflex) Agent



What if the vacuum agent is deprived from its location sensor?

- Let's write the algorithm for the vacuum cleaner …
- Percepts: location and status (location sensor, dirt sensor)
- Actions: Left, Right, Suck, NoOp

Percept	Action
[A, clean]	Right
[A, dirty]	Suck
[B, clean]	Left
[B, dirty]	Suck

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

# of possibilities

$$4^T \rightarrow 4$$

1. Simple Reflex Agents

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

- Pros:
  - Simple
- Cons:
  - For this to produce rational agents the environment must be _____?
  - Often yield infinite loops
    - Randomization can help but not so much

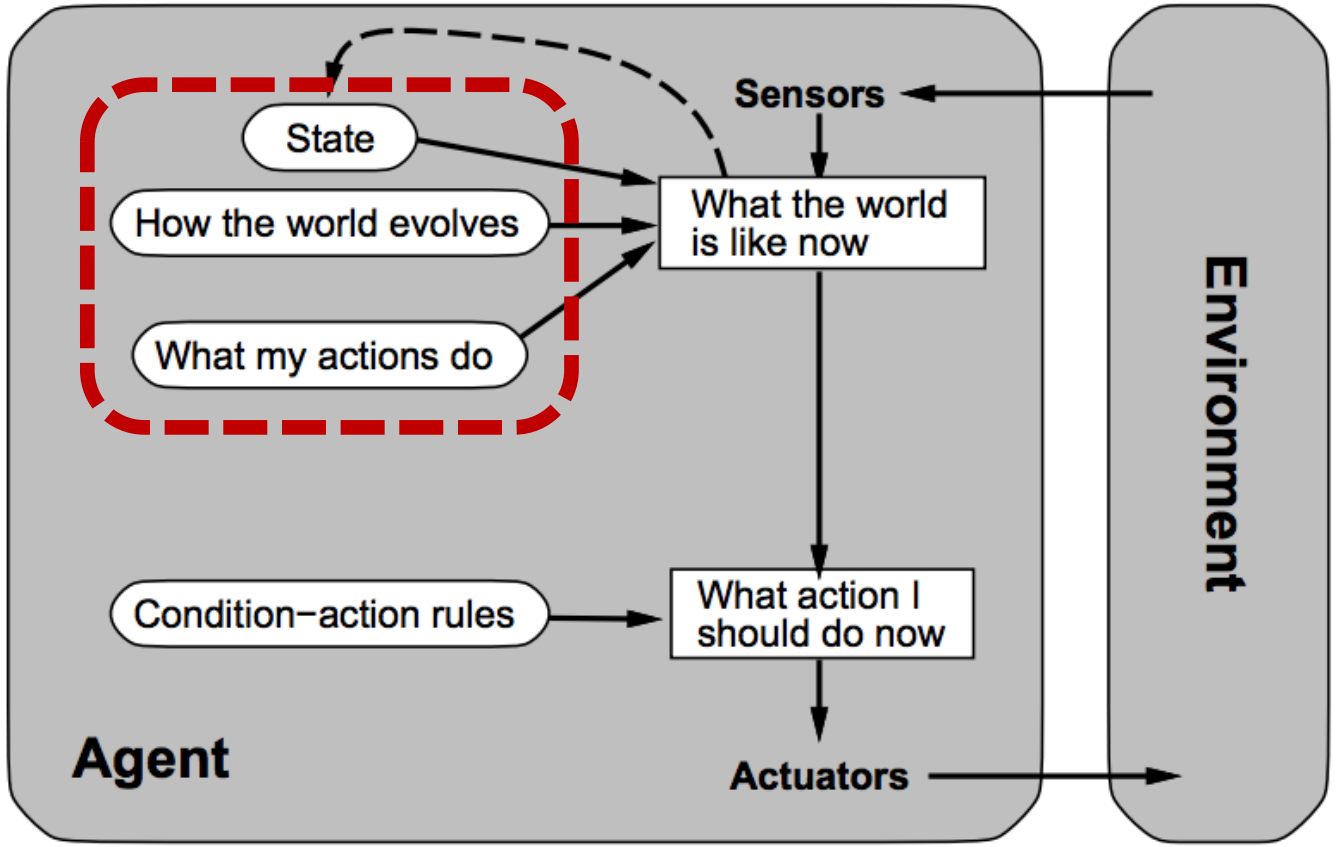
2. Model-based Reflex Agents

- That didn’t turn out so well
- Let’s try **keeping track of** what we can’t currently observe
- **Internal state** to track
  - Is that car braking?
  - Is there a car in my blind spot??
  - Where did I park???

2. Model-based Reflex Agents

"Agents that keep track of the world"

- The most effective way to **handle partial observability** is for the agent to **"keep track of the part of the world it can't see now"**
- The agent should **maintain** some sort of **internal state** that depends on the percept history (*best guess*) and reflects at least some of the unobserved aspects of the current state



2. Model-based Reflex Agents

- Updating the internal state information as time goes by requires two kinds of knowledge to be encoded in the agent program

- In other words, the environment/world is modeled based on
  1) Information about ***how the world evolves independently of the agent***
  2) Information about ***how the agent's own actions affects the world***

- This knowledge about "*how the world works*" is called a **model of the world**

2. Model-based Reflex Agents

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              model, a description of how the next state depends on current state and action
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

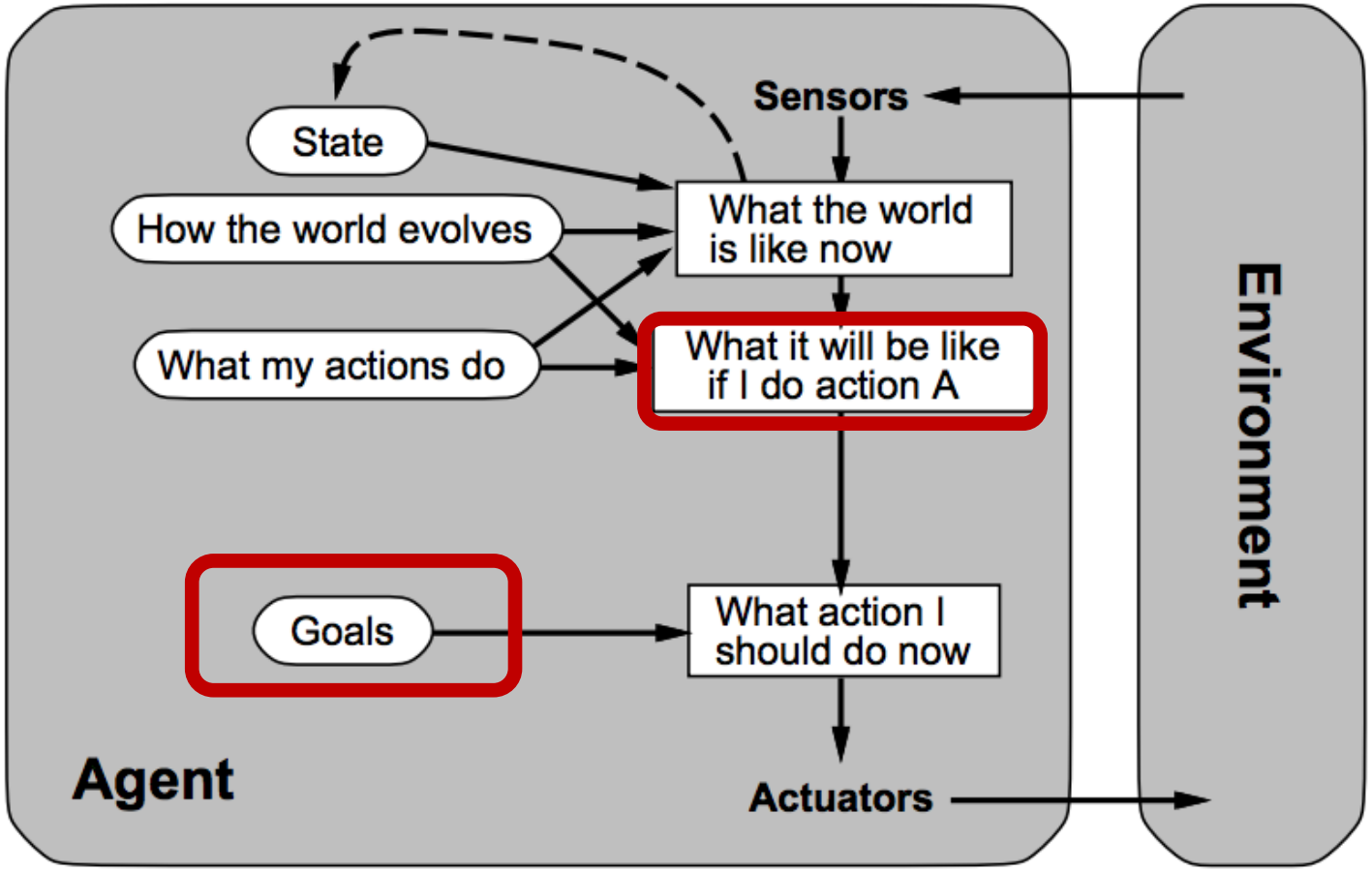
- Getting better!
- But sometimes model is insufficient:
  - Impossible to model hidden world accurately and reliably
  - What to do when environment doesn't imply action?
    - i.e., fork in the road

3. Goal-based Agents

- An improvement over model-based agents, and used in cases where knowing the current state of the environment is not enough
  - Additional to Model-based agents, these agents need some **goal information**
- Agents combine the provided goal information with the environment model to choose the actions which achieve that goal
  - Goals decide the actions of the agent
- Agents will account for the future
  - Consider the future with “What will happen if I do A?”

3. Goal-based Agents

- Knowing about the current state of the environment is not always enough to decide what to do (e.g., decision at a road junction)
- The agent needs some sort of **goal** information that **describes situations that are desirable**
- The agent program can combine this with information about the results of possible actions in order to **choose actions that achieve the goal**
- Usually requires **search** and **planning**



3. Goal-based Agents

- The goal-based decision making is different from the condition-action rules in that **the future is considered**
  - What will happen if I do that?
  - Will that make me satisfied?
- Goal-based agents vs reflex-based agents
  - The reflex agent brakes when it sees brake lights
  - A goal-based agent could reason that if the car in front has its brake lights on, it will slow down. Given the way the world usually evolves, the only action that will achieve the goal of not hitting other cars is to brake.

Goal-based Agents vs Reflex-based Agents

- Although the goal-based agent appears less efficient, it is more flexible because **“the knowledge that supports its decisions is represented explicitly and can be modified”**
  - On the other hand, for the reflex-agent, we would have to rewrite many condition-action rules
- The goal-based agent’s behavior can easily be changed to go to a different location, but the reflex agent’s rules work only for a single destination
  - The reflex agent's rules must be changed for a new situation
- Sometimes goal-based selection is not trivial when the agent must find a long sequence of actions to achieve the goal
  - Search and planning are devoted to this

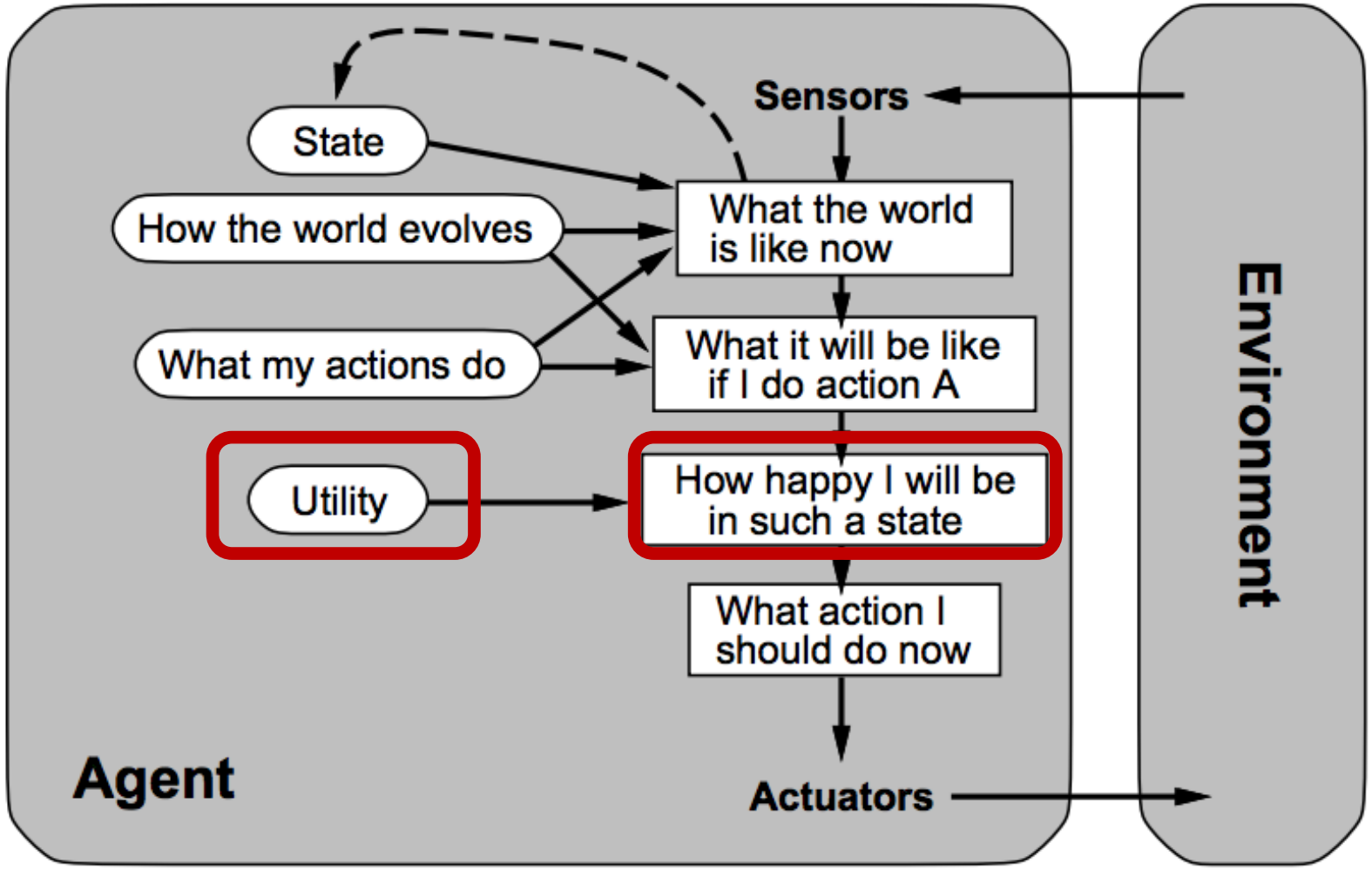
4. Utility-based Agents

- Sometimes achieving the desired goal is not enough to generate high-quality behavior in most environments
  - Goals just roughly provide a binary distinction between happy and unhappy states
  - Did you meet it or not?
- We may look for quicker, safer, cheaper trip to reach a destination
- A more general performance measure should allow a comparison of different world states according to exactly **how happy** they would make the agent if they could be achieved
  - Agent **happiness** should be taken into consideration → we call it **utility**
  - A **utility function** is the agent's performance measure – maps a state onto a real number which describes the associated degree of happiness
- Because of the uncertainty in the world, a utility agent chooses the action that maximizes the expected (average) utility

4. Utility-based Agents

Needed when:

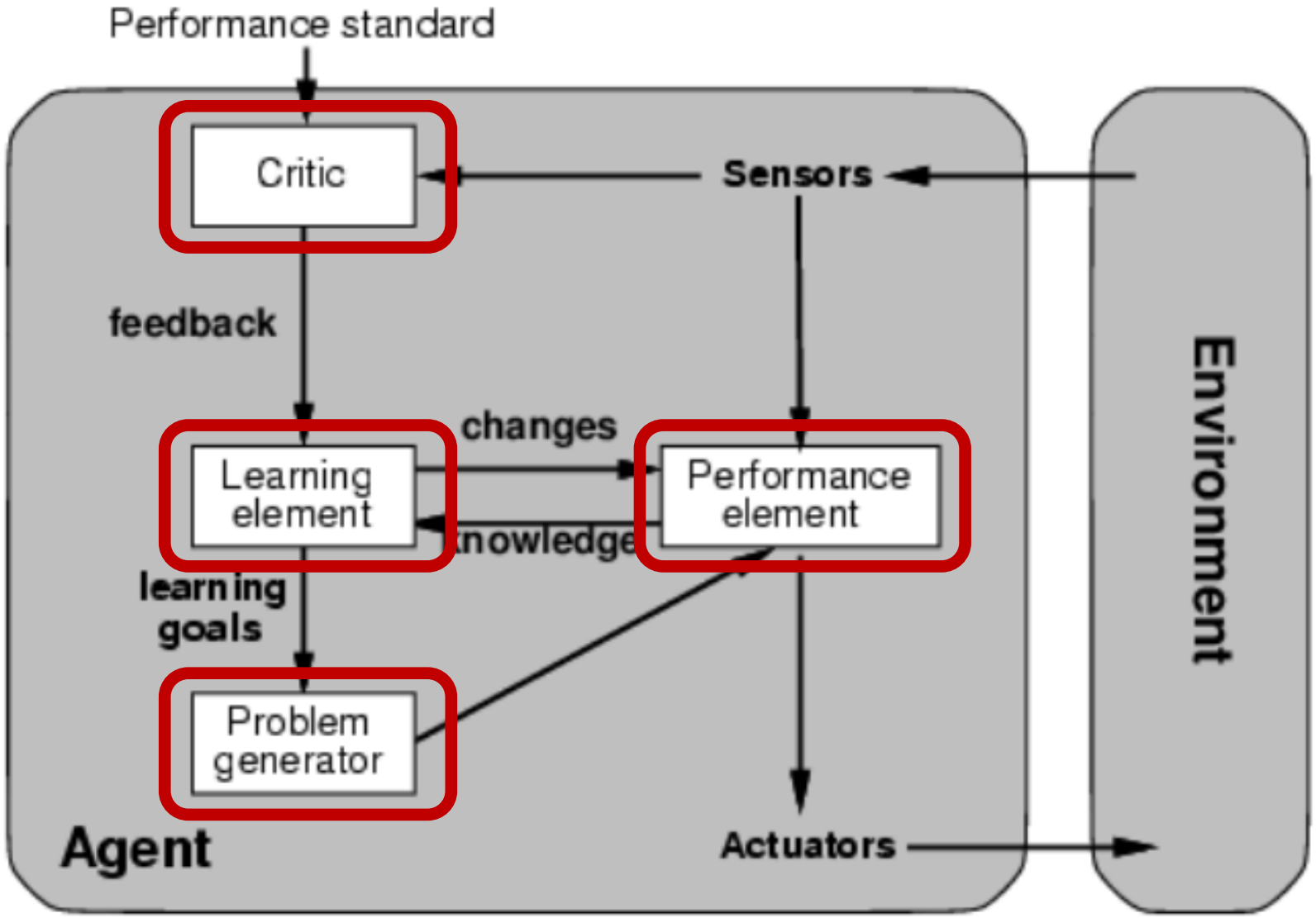
- Goals conflict
  - tradeoff between conflicting objectives (speed vs. safety)
- Uncertain goals
  - likelihood of success



Learning Agents

- That's nice – but how do we program these thing?
- By hand?
  - Manual programming is too slow and tiring
  - "Some more expeditious method seem desirable", Alan Turing, 1950
- Let the agent figure things out by itself!
- A learning agent
  - This generalized all the previous agents we have seen

Learning Agents

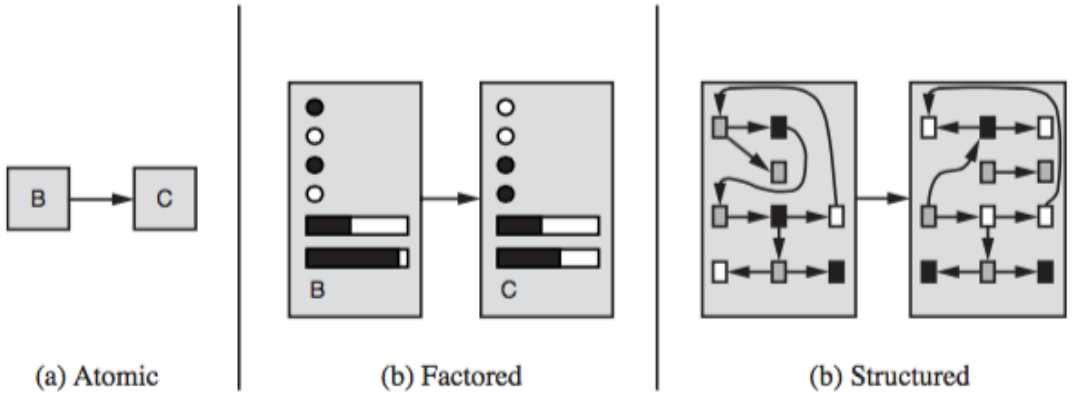


Learning Agents

- A learning agent has four conceptual components
  - **Learning Element**: responsible for making improvements to performance element
  - **Performance Element**: responsible for selecting external actions
    - It is what we considered as agent so far
  - **Critic**: How well is the agent doing w.r.t. a fixed performance standard
    - Learning element uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future
  - **Problem Generator**: suggests innovative actions
    - Responsible for suggesting actions that will lead to new and informative experiences
- Learning is a process of modification of each component to bring them into closer agreement with the available feedback information

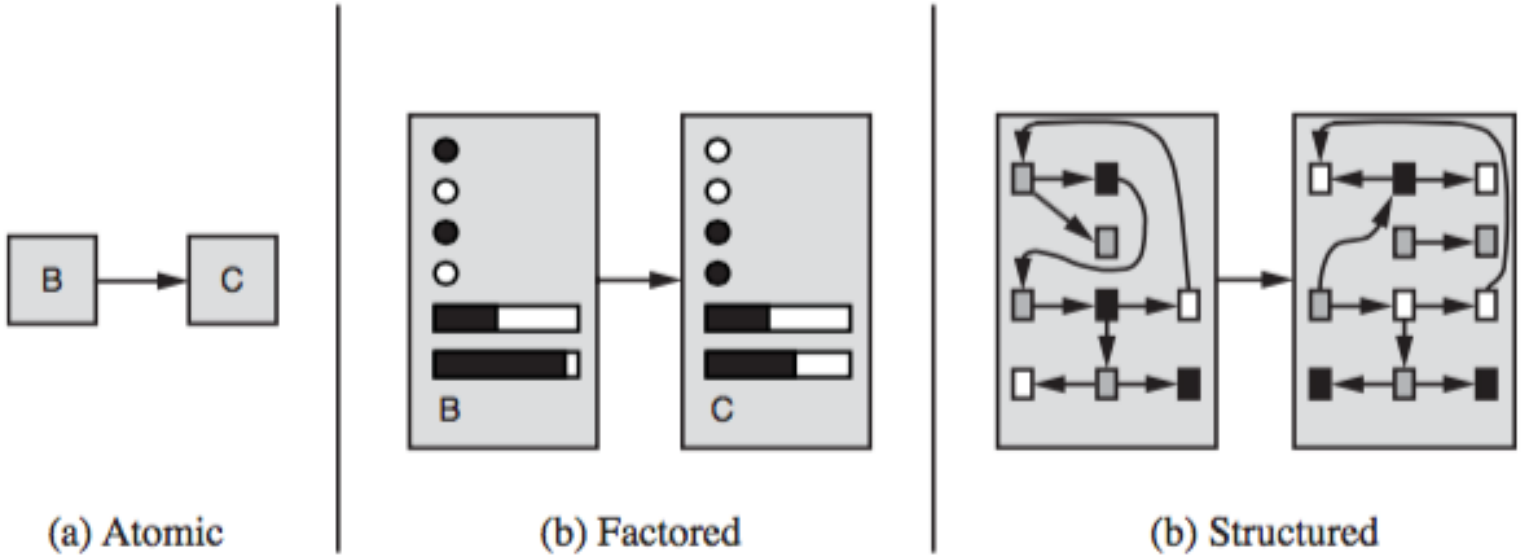
How the components of agent programs work?

- We have described agent programs (in very high-level terms) as consisting of various components, whose function is to answer questions such as:
  - "What is the world like now?"
  - "What action should I do now?"
  - "What my actions do?"
- The next question for a student of AI is
  - "How on earth do these components work?"



- Roughly speaking, we can place the representations along an axis of increasing complexity and expressive power — **atomic**, **factored**, and **structured**

How the components of agent programs work?



Three ways to represent states and the transitions between them

a) **Atomic Representation:** a state (such as B or C) is a black box with no internal structure

b) **Factored Representation:** a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols

c) **Structured Representation:** a state includes objects, each of which may have attributes of its own as well as relationships to other objects

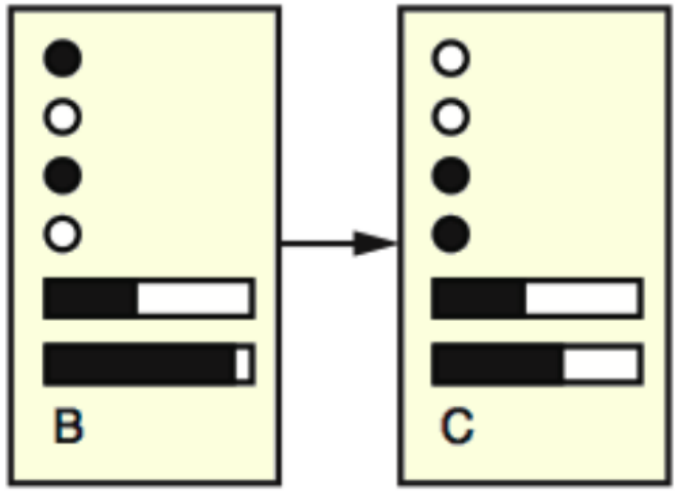
Atomic Representation

- Each state of the world is indivisible — is stored as a ***black-box***, i.e., *without any internal structure*
- ***Example:*** finding a driving route, each state is a city
  - Consider the problem of finding a driving route from one end of a country to the other via some sequence of cities
  - For the purposes of solving this problem, reduce the state of world to just the name of the city we are in—a single atom of knowledge; a "black box" whose only discernible property is that of being identical to or different from another black box
- Atomic representation works for **model-based** and **goal-based** agents
- Used in various **AI Algorithms**
  - Search
  - Game-playing
  - Hidden Markov models
  - Markov decision processes

all work with atomic representations
— or, at least, they treat representations
as if they were atomic

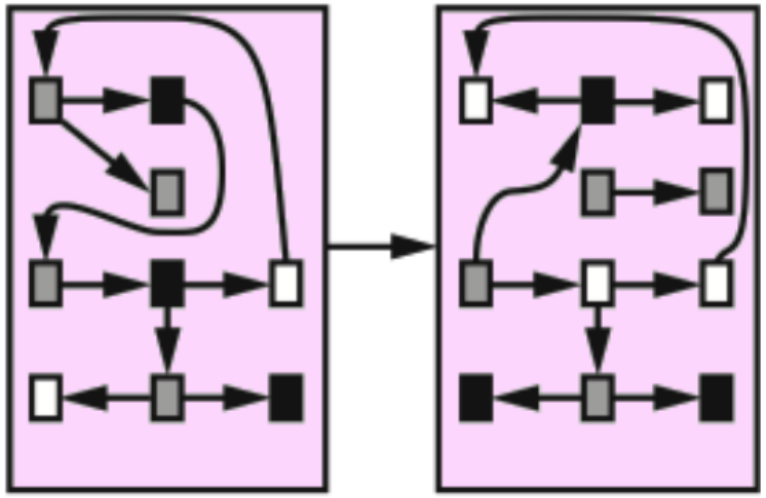
Factored Representation

- In factored representation, the state is no longer a black box. Now, ***each state has some attribute-value pairs, also known as variables that can contain a value***
  - A **factored representation** splits up each state into a fixed set of **variables** or **attributes**, each of which can have a **value**
  - While two different atomic states have nothing in common—they are just different black boxes—two different factored states can share some attributes (such as being at some particular GPS location) and not others (such as having lots of gas or having no gas); this makes it much easier to work out how to turn one state into another
- *Example:* While finding a route, you have a GPS location and amount of gas in the tank – adds a constraint to the problem
- With factored representations, we can also represent ***uncertainty***
  - Ignorance about the amount of gas in the tank can be represented by leaving that attribute blank
- Factored representation works for **goal-based** agents
- Many important areas of AI are based on factored representation
  - Constraint Satisfaction algorithms
  - Propositional Logic
  - Planning
  - Bayesian networks
  - Machine Learning algorithms



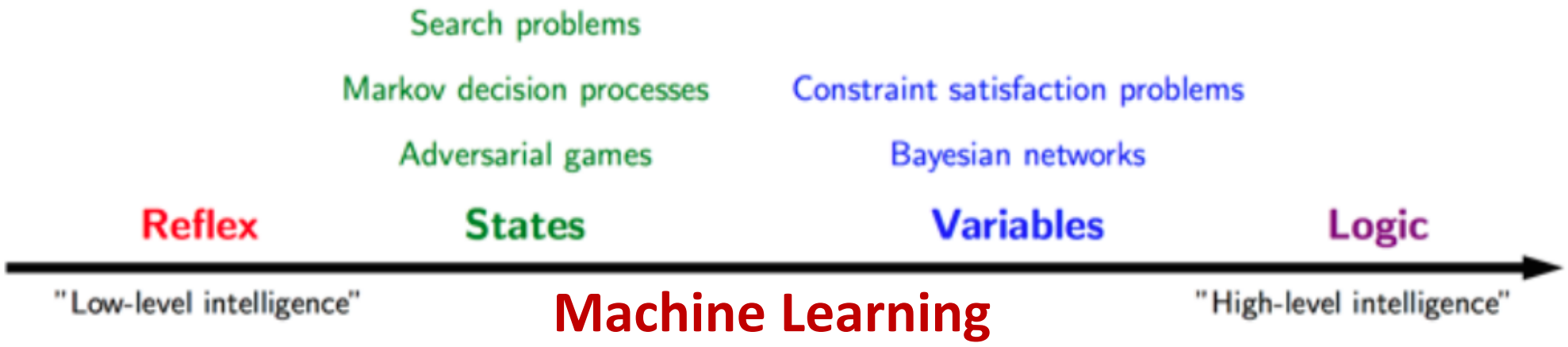
Structured Representation

- For many purposes, we need to understand the world as having ***things*** in it that are ***related*** to each other, not just variables with values
  - For example, we might notice that a large truck ahead of us is reversing into the driveway of a dairy farm but a cow has got loose and is blocking the truck's path
  - A factored representation is unlikely to be pre-equipped with the attribute ***TruckAheadBackingIntoDairyFarmDrivewayBlockedByLooseCow*** with value ***true*** or ***false***
  - Instead, we would need a **structured representation**, in which **objects** such as cows and trucks and their various **relationships** can be described explicitly
- In structured representation, ***we have relationships between the variables/factored states - This induces logic in the AI algorithms***
  - For example, in natural language processing, the states are whether the statement contains a reference to a person and whether the adjective in that statement represents that person. The relation in these states will decide, whether the statement was a sarcastic one
- High level AI, used in algorithms like
  - First-order logic
  - First-order probability models
  - Knowledge-based learning
  - Natural language understanding



Representation of States: Atomic, Factored, Structured

- The axis along which atomic, factored, and structured representations lie is the axis of increasing **expressiveness** power
- A more expressive representation can capture, at least as concisely, everything a less expressive one can capture, plus some more
- Often, the more expressive language is *much* more concise



Credit: Courtesy Percy Liang

Summary

- The concept of intelligent agent is central in AI
- AI aims to design intelligent agents that are useful, reactive, autonomous, and even social and pro-active
- An **agent perceives** its environment through **percepts**, and **acts** through **actuators**
- A **performance measure** evaluates the behavior of the agent
- An agent that acts to maximize its expected **performance measure** given the **percept** sequence and candidate **actions,** is called a **rational agent**
- We can define different criteria for rational agent environment, which is called **PEAS** -- a task environment specification that includes **P**erformance measure, **E**nvironment, **A**ctuators, and **S**ensors
- Task environments vary along several dimensions
- An agent is a set of **architecture**, or **hardware**, plus a **program** or **system**, where both must be compatible to work together

Agent = Architecture + Program

Summary

- There exists four types of agents
  - Simple reflex agents – that are the simplest ones
  - Model-based agents – that try to model the world by creating the model internally
  - Goal-based agents – that wants to model the world, but still have a goal to achieve
  - Utility-based agents – that try to increase their happiness
- All agents can improve their performance through ***learning***
  - ***which is a central piece today in AI that aims to make the agent aware of different situations, and learn from its experience***
- This was a high-level representation of the different agents' programs
- Lastly, we see that state representation can be either **atomic**, **factored**, or **structured**, which goes by <u>increasing order of expressiveness</u>

Next Week: Search Agents

- Agents that work towards a **goal**
- Agents consider the impact of **actions** on future **states**
- Agent's job is to identify the action or series of actions that lead to the goal
- Paths come with different costs and depths
- Two kinds of search
  - **Uninformed Search** (use no domain knowledge): BFS, DFS, UCS, etc.
  - **Informed Search** (use heuristic to reach the goal faster): Greedy search, A*, etc.